

KONGU ENGINEERING COLLEGE
(AUTONOMOUS)

PERUNDURAI, ERODE – 638 060

EXPERIMENT - 10

PROJECT TITLE

**INDUSTRIAL DG MONITORING AND ANALYTICS
SYSTEM**

SUBMITTED BY

ABISHEK R S - 24MER002

GOWRISANKAR K - 24MER010

NAVEENRAJ K - 24MER035

PRAVEEN N M - 24MER043

INDUSTRIAL DG MONITORING AND ANALYTICS SYSTEM

1. Project review:

The Industrial DG Monitoring and Analytics System is designed to continuously monitor the operating condition and performance of an industrial diesel generator (DG). The system uses an ESP8266 NodeMCU as the central controller along with sensors to monitor important parameters such as temperature and vibration.

The sensor readings are continuously collected and processed by the ESP8266. The measured values are compared with predefined operating limits to identify abnormal conditions. When high temperature or abnormal vibration is detected, the system generates an alert using a buzzer and LED. A relay module can also be used to demonstrate generator protection or automatic shutdown.

The collected data can be displayed and analyzed to understand the operating condition of the generator. Since the ESP8266 has built-in Wi-Fi capability, the system can be extended for IoT-based remote monitoring, cloud data storage, performance analysis, maintenance alerts, and predictive maintenance.

2. Problem Statement:

Industrial diesel generators are widely used as backup and primary power sources. During continuous operation, the generator may experience problems such as overheating, excessive vibration, and abnormal operating conditions.

Manual monitoring does not provide continuous information about the generator's condition. Failure to identify abnormal conditions at an early stage may result in equipment damage, unexpected shutdown, increased maintenance cost, and interruption of power supply.

Therefore, there is a need for an automatic system that can continuously monitor important generator parameters, analyze the operating condition, and provide an early warning when abnormal conditions occur.

The proposed system addresses this problem using sensors, ESP8266 NodeMCU, buzzer, LED, and relay module for continuous monitoring and basic performance analytics.

3. Proposed Solution:

The proposed system consists of an ESP8266-based Industrial DG Monitoring and Analytics System.

A temperature sensor monitors the operating temperature of the generator, while a vibration sensor detects abnormal vibration. The sensor data is continuously collected by the ESP8266 NodeMCU.

The ESP8266 processes and analyzes the sensor readings by comparing them with predefined limits. If the measured values exceed the safety limits, the system activates the buzzer, LED, and relay to indicate an abnormal generator condition.

Proposed Operation:

1. Start the system and initialize the ESP8266.
2. Initialize the temperature and vibration sensors.
3. Read the generator temperature.
4. Read the generator vibration condition.
5. Process the sensor readings.
6. Compare the readings with predefined limits.
7. Analyze the generator operating condition.
8. If an abnormal condition is detected, activate the alert system.
9. Turn ON the buzzer and LED.
10. Activate the relay for protection or shutdown.
11. Display the generator status and maintenance information.
12. Repeat the monitoring and analytics process continuously.

4. System Architecture:

The Industrial DG Monitoring and Analytics System consists of the following sections:

Input Section:

Temperature Sensor – Measures the generator operating temperature.

Vibration Sensor – Detects abnormal generator vibration. The sensors send the measured data to the ESP8266 NodeMCU.

Processing Section:

The ESP8266 NodeMCU acts as the central controller.

- Receives sensor data.
- Processes temperature and vibration readings.
- Compares values with predefined limits.
- Identifies abnormal conditions.
- Generates basic generator health information.
- Controls the output devices.

Actuation Section:

The relay module is used for generator protection.

- Receives the control signal from ESP8266.
- Can be used to demonstrate generator shutdown.
- Can control an external warning or protection system.

Indication Section:

The buzzer and LED provide warning indications.

- Buzzer OFF and LED OFF → DG condition normal.
- Buzzer ON and LED ON → Abnormal DG condition detected.

Connectivity Section:

The ESP8266 provides Wi-Fi capability for future expansion.

The system can be developed for:

- Remote DG monitoring.
- Cloud-based data storage.
- Performance analytics.

- Maintenance history.
- Real-time alerts.
- Predictive maintenance.

5. Circuit Table:

Important Pin Mapping:

ESP8266 NodeMCU Pin Component

A0	A0 Temperature Sensor → Analog Output
GPIO 2	D4 Vibration Sensor → OUT
GPIO 14	D5 Relay Module → IN
GPIO 12	D6 Buzzer → Positive Terminal
GPIO 13	D7 LED → Anode through resistor
3.3V / 5V	– Sensor Power
GND	– Common Ground

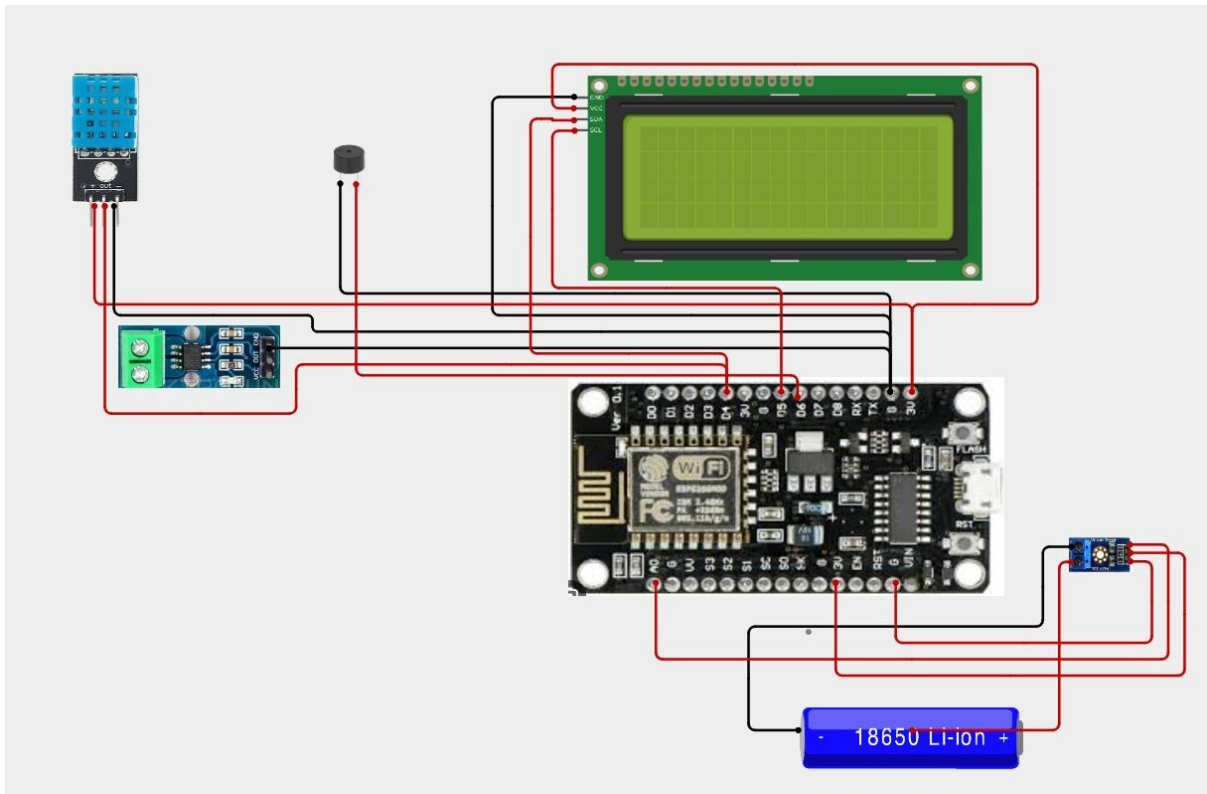
6. Circuit Design:

The circuit consists of an ESP8266 NodeMCU, temperature sensor, vibration sensor, relay module, buzzer, LED, breadboard, and jumper wires.

The temperature sensor is connected to the A0 input of the ESP8266 to monitor the generator operating temperature. The vibration sensor output is connected to GPIO2 (D4) to detect abnormal generator vibration.

The relay module is connected to GPIO14 (D5). The buzzer is connected to GPIO12 (D6), and the LED is connected to GPIO13 (D7) through a suitable resistor.

The ESP8266 continuously reads and analyzes the sensor data. When the temperature or vibration exceeds the predefined safety limit, the controller activates the buzzer, LED, and relay.



7. Algorithm:

1. Start the system.
2. Initialize the **ESP8266 NodeMCU**, **temperature sensor**, **vibration sensor**, **relay**, **buzzer**, and **LED**.
3. Read the **temperature value** from the temperature sensor.
4. Read the **vibration status** from the vibration sensor.
5. Process and compare the sensor readings with the predefined safety limits.
6. Analyze whether the **DG condition is normal or abnormal**.
7. If high temperature or abnormal vibration is detected, turn **ON the buzzer, LED, and relay**.
8. Display **“DG STATUS: ABNORMAL – MAINTENANCE REQUIRED”**.
9. Otherwise, keep the **buzzer, LED, and relay OFF** and display **“DG STATUS: NORMAL – HEALTHY OPERATION”**.
10. Wait for a short interval and **repeat the monitoring and analytics process continuously**.

8. Coding:

```
#include <ESP8266WiFi.h>
```

```
#include <ThingSpeak.h>
```

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
#include <DHT.h>
```

```
// =====
```

```
// CLOUD-BASED INDUSTRIAL DIESEL GENERATOR
```

```
// MONITORING AND ANALYTICS SYSTEM
```

```
//
```

```
// ESP8266 NodeMCU
```

```
// DHT11
```

```
// Voltage Sensor
```

```
// ACS712
```

```
// OLED
```

```
// Buzzer
```

```
// WiFi + ThingSpeak Cloud
```

```
// =====
```

```
// =====
```

```
// WIFI
```

```
// =====
```

```
const char* WIFI_SSID = "praveen";
```

```
const char* WIFI_PASSWORD = "praveen12345";
```

```
// =====
```

```
// THINGSPEAK
```

```
// =====
```

```
const char* API_KEY = "0IP4GOX0SMNK3JBV";
```

```
// CHANGE THIS
```

```
unsigned long CHANNEL_ID = 1234567;
```

```
// =====
```

```
// DHT11
```

```
// =====
```

```
#define DHT_PIN 2
```

```
#define DHT_TYPE DHT11
```

```
DHT dht(
```

```
    DHT_PIN,
```

```
    DHT_TYPE
```

```
);
```



```
// =====
```

```
// VOLTAGE SENSOR
```

```
// =====
```

```
#define VOLTAGE_PIN A0
```

```
// =====
```

```
// BUZZER
```

```
// =====
```

```
#define BUZZER_PIN 12
```

```
// D6 = GPIO12
```

```
// =====
```

```
// OLED
```

```
// =====
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
#define OLED_RESET -1
```

```
#define OLED_ADDRESS 0x3C
```

```
Adafruit_SSD1306 display(  
  SCREEN_WIDTH,  
  SCREEN_HEIGHT,  
  &Wire,  
  OLED_RESET  
);
```

```
// =====
```

```
// WIFI CLIENT
```

```
// =====
```

```
WiFiClient client;
```

```
// =====
```

```
// VOLTAGE CALIBRATION
```

```
// =====
```

```
// For a common low-voltage 0-25V module,
```

```
// this is only a starting value.
```

```
//
```

```
// CALIBRATE using a multimeter.
```

```
float voltageCalibration = 5.0;
```

```
// =====
```

```
// GENERATOR LIMITS
```

```
// =====
```

```
float minimumVoltage = 200.0;
```

```
float maximumVoltage = 250.0;
```

```
float maximumTemperature = 50.0;
```

```
float maximumHumidity = 85.0;
```

```
// =====
```

```
// CLOUD UPDATE INTERVAL
```

```
// =====
```

```
unsigned long lastThingSpeakUpdate = 0;
```

```
const unsigned long CLOUD_INTERVAL = 20000;
```

```
// =====
```

```
// WIFI RECONNECT TIMER
```

```
// =====
```

```
unsigned long lastWiFiCheck = 0;
```

```
const unsigned long WIFI_CHECK_INTERVAL = 10000;
```

```
// =====
```

```
// SENSOR VALUES
```

```
// =====
```

```
float generatorVoltage = 0;
```

```
float temperature = 0;
```

```
float humidity = 0;
```

```
// =====
```

```
// STATUS
```

```
// =====
```

```
bool alarm = false;
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  delay(1000);
```

```
  // =====
```

```
  // I2C
```

```
  // =====
```

```
  // SDA = D2 / GPIO4
```

```
  // SCL = D1 / GPIO5
```

```
  Wire.begin(D2, D1);
```

```
  // =====
```

```
  // DHT
```

```
  // =====
```

```
  dht.begin();
```

```
  // =====
```

```
  // BUZZER
```

```
//=====
```

```
pinMode(  
  BUZZER_PIN,  
  OUTPUT  
);
```

```
digitalWrite(  
  BUZZER_PIN,  
  LOW  
);
```

```
//=====
```

```
// OLED
```

```
//=====
```

```
if(  
  !display.begin(  
    SSD1306_SWITCHCAPVCC,  
    OLED_ADDRESS  
  )  
)  
{  
  Serial.println(  
    "OLED ERROR!"  
  );  
}
```

```
while (true)
{
    delay(100);
}
}

// =====
// STARTUP DISPLAY
// =====

display.clearDisplay();

display.setTextColor(
    SSD1306_WHITE
);

display.setTextSize(1);

display.setCursor(0, 0);

display.println(
    "INDUSTRIAL DG"
);

display.setCursor(0, 12);
```

```
display.println(  
  "CLOUD MONITORING"  
);
```

```
display.setCursor(0, 24);
```

```
display.println(  
  "SYSTEM"  
);
```

```
display.setCursor(0, 42);
```

```
display.println(  
  "Initializing..."  
);
```

```
display.display();
```

```
delay(2000);
```

```
// =====  
// WIFI  
// =====
```



```
connectWiFi();
```

```
// =====
```

```
// THINGSPEAK
```

```
// =====
```

```
ThingSpeak.begin(client);
```

```
Serial.println();
```

```
Serial.println(
```

```
    "=====
```

```
);
```

```
Serial.println(
```

```
    "CLOUD DG MONITORING READY"
```

```
);
```

```
Serial.println(
```

```
    "=====
```

```
);
```

```
}
```

```
// =====
```

```
// WIFI CONNECTION
```

```
// =====
```

```
void connectWiFi()
```

```
{
```

```
    Serial.println();
```

```
    Serial.println(
```

```
        "Connecting to WiFi..."
```

```
    );
```

```
    WiFi.mode(WIFI_STA);
```

```
    WiFi.begin(
```

```
        WIFI_SSID,
```

```
        WIFI_PASSWORD
```

```
    );
```

```
    int attempts = 0;
```

```
    while (
```

```
        WiFi.status() != WL_CONNECTED &&
```

```
        attempts < 30
```

```
    )
```

```
{  
  delay(500);  
  
  Serial.print(".");  
  
  attempts++;  
}
```

```
Serial.println();
```

```
if(  
  WiFi.status() == WL_CONNECTED  
)  
{  
  Serial.println(  
    "WiFi CONNECTED"  
  );
```

```
  Serial.print(  
    "IP Address: "  
  );
```

```
  Serial.println(  
    WiFi.localIP()  
  );
```

```
}  
else  
{  
  Serial.println(  
    "WiFi CONNECTION FAILED"  
  );  
}  
}
```

```
// =====  
// READ VOLTAGE  
// =====
```

```
float readVoltage()  
{  
  int rawValue =  
    analogRead(  
      VOLTAGE_PIN  
    );
```

```
float adcVoltage =  
  (rawValue / 1023.0) * 3.3;
```

```
float voltage =
```

```
adcVoltage *  
voltageCalibration;
```

```
return voltage;  
}
```

```
// =====  
// READ ALL SENSORS  
// =====
```

```
void readSensors()
```

```
{  
    // Voltage
```

```
    generatorVoltage =  
        readVoltage();
```

```
    // Temperature
```

```
    temperature =  
        dht.readTemperature();
```

```
    // Humidity
```

```
humidity =  
    dht.readHumidity();
```

```
// DHT error
```

```
if(  
    isnan(temperature) ||  
    isnan(humidity)  
)  
{  
    Serial.println(  
        "DHT11 SENSOR ERROR"  
    );  
  
    return;  
}  
}
```

```
// =====  
// ANALYZE GENERATOR  
// =====
```

```
void analyzeGenerator()  
{
```

```
alarm = false;
```

```
// -----
```

```
// LOW VOLTAGE
```

```
// -----
```

```
if (
```

```
    generatorVoltage <
```

```
    minimumVoltage
```

```
)
```

```
{
```

```
    alarm = true;
```

```
    Serial.println(
```

```
        "ANALYTICS: LOW VOLTAGE"
```

```
    );
```

```
}
```

```
// -----
```

```
// HIGH VOLTAGE
```

```
// -----
```

```
if (
```

```
    generatorVoltage >
```

```
    maximumVoltage
```

```
)  
{  
  alarm = true;  
  
  Serial.println(  
    "ANALYTICS: HIGH VOLTAGE"  
  );  
}
```

```
// -----  
// HIGH TEMPERATURE  
// -----
```

```
if(  
  temperature >  
  maximumTemperature  
)  
{  
  alarm = true;  
  
  Serial.println(  
    "ANALYTICS: HIGH TEMPERATURE"  
  );  
}
```



```
// -----  
// HIGH HUMIDITY  
// -----
```

```
if(  
    humidity >  
    maximumHumidity  
)  
{  
    alarm = true;  
  
    Serial.println(  
        "ANALYTICS: HIGH HUMIDITY"  
    );  
}
```

```
// -----  
// BUZZER  
// -----
```

```
if(alarm)  
{  
    digitalWrite(  
        BUZZER_PIN,  
        HIGH  
    );  
}
```

```
}  
else  
{  
    digitalWrite(  
        BUZZER_PIN,  
        LOW  
    );  
}  
}  
  
// =====  
// OLED DISPLAY  
// =====  
  
void updateOLED()  
{  
    display.clearDisplay();  
  
    display.setTextSize(1);  
  
    display.setTextColor(  
        SSD1306_WHITE  
    );  
}
```

```
// -----  
// VOLTAGE  
// -----
```

```
display.setCursor(0, 0);
```

```
display.print(  
  "V: "  
);
```

```
display.print(  
  generatorVoltage,  
  1  
);
```

```
display.println(  
  " V"  
);
```

```
// -----  
// TEMPERATURE  
// -----
```

```
display.setCursor(0, 15);
```

```
display.print(  
  "
```

```
"T: "
```

```
);
```

```
display.print(  
  temperature,
```

```
  1
```

```
);
```

```
display.println(  
  " C"
```

```
);
```

```
// -----
```

```
// HUMIDITY
```

```
// -----
```

```
display.setCursor(0, 30);
```

```
display.print(  
  "H: "
```

```
);
```

```
display.print(  
  humidity,
```

```
  1
```

```
);
```

```
display.println(
```

```
    "%"
```

```
);
```

```
// -----
```

```
// SYSTEM STATUS
```

```
// -----
```

```
display.setCursor(0, 45);
```

```
if (alarm)
```

```
{
```

```
    display.println(
```

```
        "STATUS: WARNING"
```

```
    );
```

```
}
```

```
else
```

```
{
```

```
    display.println(
```

```
        "STATUS: NORMAL"
```

```
    );
```

```
}
```

```
display.display();
```

```
}
```

```
// =====
```

```
// SERIAL OUTPUT
```

```
// =====
```

```
void printSerialData()
```

```
{
```

```
    Serial.println();
```

```
    Serial.println(
```

```
        "=====
```

```
);
```

```
    Serial.println(
```

```
        "INDUSTRIAL DG STATUS"
```

```
);
```

```
    Serial.println(
```

```
        "=====
```

```
);
```

```
    Serial.print(
```

```
        "Voltage  : "
```

```
);
```

```
Serial.print(  
    generatorVoltage,  
    2  
);
```

```
Serial.println(  
    " V"  
);
```

```
Serial.print(  
    "Temperature : "  
);
```

```
Serial.print(  
    temperature,  
    2  
);
```

```
Serial.println(  
    " C"  
);
```

```
Serial.print(  
    "Humidity  : "  
);
```

```
);
```

```
Serial.print(
```

```
    humidity,
```

```
    2
```

```
);
```

```
Serial.println(
```

```
    "%0"
```

```
);
```

```
if (alarm)
```

```
{
```

```
    Serial.println(
```

```
        "Condition : WARNING"
```

```
);
```

```
Serial.println(
```

```
    "Buzzer : ON"
```

```
);
```

```
}
```

```
else
```

```
{
```

```
    Serial.println(
```

```
        "Condition : NORMAL"
```

```
);
```



```
Serial.println(  
    "Buzzer    : OFF"  
);  
}
```

```
if(  
    WiFi.status() ==  
    WL_CONNECTED  
)  
{  
    Serial.println(  
        "Cloud    : CONNECTED"  
    );  
}  
else  
{  
    Serial.println(  
        "Cloud    : DISCONNECTED"  
    );  
}
```

```
Serial.println(  
    "===== "  
);
```

```
}
```

```
// =====
```

```
// SEND DATA TO THINGSPEAK
```

```
// =====
```

```
void sendToThingSpeak()
```

```
{
```

```
  if (
```

```
    WiFi.status() !=
```

```
    WL_CONNECTED
```

```
  )
```

```
  {
```

```
    Serial.println(
```

```
      "Cloud upload skipped - WiFi offline"
```

```
    );
```

```
    return;
```

```
  }
```

```
Serial.println();
```

```
Serial.println(
```

```
  "Uploading data to ThingSpeak..."
```

```
);
```

```
// Field 1
```

```
// Generator Voltage
```

```
ThingSpeak.setField(  
    1,  
    generatorVoltage  
);
```

```
// Field 2
```

```
// Temperature
```

```
ThingSpeak.setField(  
    2,  
    temperature  
);
```

```
// Field 3
```

```
// Humidity
```

```
ThingSpeak.setField(  
    3,  
    humidity  
);
```

```
// Field 4
```

```
// Alarm
```

```
int alarmStatus =  
  alarm ? 1 : 0;
```

```
ThingSpeak.setField(  
  4,  
  alarmStatus  
);
```

```
// Field 5
```

```
// WiFi signal strength
```

```
long signalStrength =  
  WiFi.RSSI();
```

```
ThingSpeak.setField(  
  5,  
  signalStrength  
);
```

```
// Upload
```

```
int response =
```

```
    ThingSpeak.writeFields(
```

```
        CHANNEL_ID,
```

```
        API_KEY
```

```
    );
```

```
if (
```

```
    response == 200
```

```
)
```

```
{
```

```
    Serial.println(
```

```
        "CLOUD UPLOAD SUCCESS"
```

```
    );
```

```
}
```

```
else
```

```
{
```

```
    Serial.print(
```

```
        "ThingSpeak ERROR: "
```

```
    );
```

```
Serial.println(
```

```
    response
```

```
);
```

```
}  
}
```

```
// =====  
// WIFI MONITOR  
// =====
```

```
void checkWiFi()  
{  
  if(  
    millis() -  
    lastWiFiCheck >=  
    WIFI_CHECK_INTERVAL  
  )  
  {  
    lastWiFiCheck =  
      millis();  
  
    if(  
      WiFi.status() !=  
      WL_CONNECTED  
    )  
    {  
      Serial.println(  
        "WiFi disconnected!"  
      );  
    }  
  }  
}
```

```
    );

    connectWiFi();
}
}
}

// =====
// MAIN LOOP
// =====

void loop()
{
    // Check WiFi

    checkWiFi();

    // Read sensors

    readSensors();

    // Analyze data

    if(
```

```
    !isnan(temperature) &&  
    !isnan(humidity)  
  )  
  {  
    analyzeGenerator();  
  }
```

```
// Serial monitor
```

```
printSerialData();
```

```
// OLED
```

```
if (  
    !isnan(temperature) &&  
    !isnan(humidity)  
)  
{  
    updateOLED();  
}
```

```
// Cloud upload
```

```
if (
```



```
    millis() -  
    lastThingSpeakUpdate >=  
    CLOUD_INTERVAL  
)  
{  
    lastThingSpeakUpdate =  
        millis();  
  
    if (  
        !isnan(temperature) &&  
        !isnan(humidity)  
    )  
    {  
        sendToThingSpeak();  
    }  
}  
  
    delay(3000);  
}
```

Coding Output:

```
sketch_aug14b
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// =====
// CLOUD-BASED INDUSTRIAL DIESEL GENERATOR
// MONITORING AND ANALYTICS SYSTEM
//
// ESP8266 NodeMCU
// DHT11
// Voltage Sensor
// ACP712
// OLED
// Buzzer
// Wifi + ThingSpeak Cloud
// =====

// =====
// WIFI
// =====

const char* WIFI_SSID = "praveen";
const char* WIFI_PASSWORD = "praveen12345";

// =====
// THINGSPEAK
// =====

const char* API_KEY = "0iP4G0X08MNX3JBV";

// CHANGE THIS
unsigned long CHANNEL_ID = 1234567;

Done uploading.
```

COM6

ANALYTICS: LOW VOLTAGE

=====

INDUSTRIAL DG STATUS

=====

Voltage : 4.00 V

Temperature : 27.20 C

Humidity : 76.80 %

Condition : WARNING

Buzzer : ON

Cloud : CONNECTED

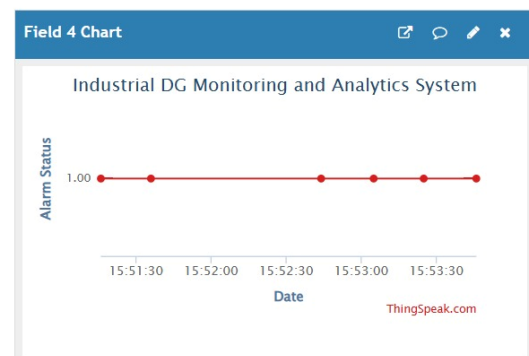
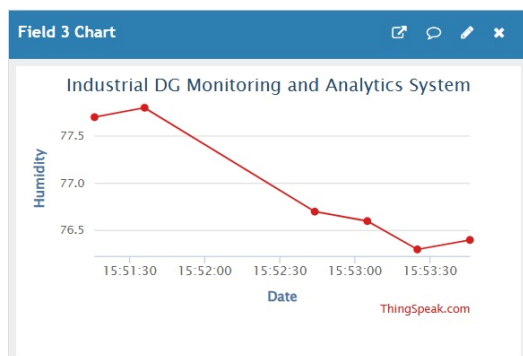
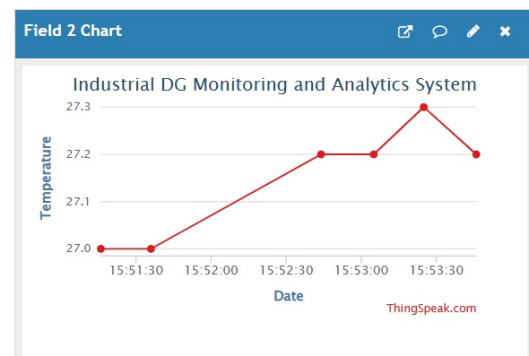
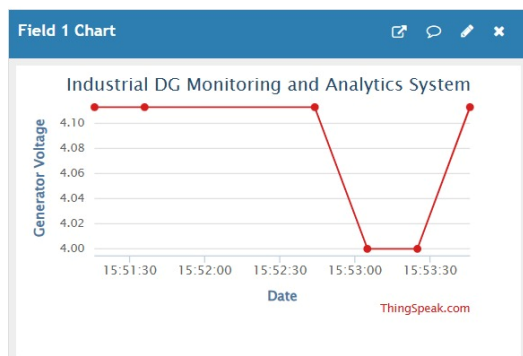
=====

☒ Autoscroll ☐ Show timestamp

Rawline

9600 baud

Clear output



9. System Working:

Power ON the System:

The ESP8266 NodeMCU is powered ON.

Initialize Components:

The temperature sensor, vibration sensor, relay, buzzer, and LED are initialized.

Monitor DG Temperature

The temperature sensor continuously monitors the generator operating temperature.

Monitor DG Vibration

The vibration sensor detects abnormal generator vibration.

Process Sensor Data

The ESP8266 receives the temperature and vibration readings.

Analyze Generator Condition

The sensor readings are compared with predefined safety limits.

Abnormal DG Condition

If high temperature or abnormal vibration is detected:

- Buzzer → ON
- LED → ON
- Relay → ON
- DG Status → Abnormal
- Analytics Result → Maintenance Required

Normal DG Condition

If the generator operates within normal limits:

- Buzzer → OFF
- LED → OFF
- Relay → OFF

- DG Status → Normal
- Analytics Result → Healthy Operation

Repeat the Process:

- The system continuously monitors and analyzes the generator operating condition.

10. Prototype Implementation:

The prototype is implemented using an ESP8266 NodeMCU, temperature sensor, vibration sensor, relay module, buzzer, LED, breadboard, and jumper wires.

The temperature sensor monitors the operating temperature of the DG, while the vibration sensor detects abnormal vibration. The ESP8266 processes the sensor data and analyzes the generator condition.

Hardware Setup

Connect the Temperature Sensor output → A0 of ESP8266.

Connect the Vibration Sensor OUT pin → D4 (GPIO2).

Connect the Relay IN pin → D5 (GPIO14).

Connect the Buzzer → D6 (GPIO12).

Connect the LED → D7 (GPIO13) through a suitable resistor.

Connect all sensor power pins to a suitable power supply.

Connect all GND terminals to a common ground.

Upload the program to the ESP8266 using the Arduino IDE.

Power ON the prototype.

Observe the temperature and vibration readings.

Create an abnormal condition by increasing temperature or vibration.

Observe the activation of the buzzer, LED, and relay.

Prototype Working:

The ESP8266 continuously collects temperature and vibration data.

The sensor readings are processed by the controller.

The measured values are compared with predefined limits.

The system analyzes the DG operating condition.

If abnormal conditions are detected, the alert system is activated.

The buzzer provides an audible warning.

The LED provides a visual warning.

The relay demonstrates generator protection or shutdown.

The Serial Monitor displays the DG status and analysis result.

The process repeats continuously for real-time monitoring

11. Bill of Materials:

S.No.	Component	Specification	Quantity
1	ESP8266	NodeMCU Wi-Fi Development Board	1
2	Temperature Sensor	Analog Temperature Sensor	1
3	Vibration Sensor	Vibration Detection Module	1
4	Relay Module	1-Channel Relay	1
5	Buzzer	3.3V/5V Buzzer	1
6	LED	Standard Indicator LED	1
7	Resistor	Suitable LED Current Limiting Resistor	1
8	Breadboard	Standard Prototype Board	1
9	Jumper Wires	Male-Male / Male-Female	1 Set

S.No.	Component	Specification	Quantity
10	USB Cable	Micro-USB	1
11	Power Supply	Suitable Regulated Supply	1

12. Results & Performance Analysis:

The prototype successfully monitors and analyzes the basic operating condition of an industrial diesel generator.

- The temperature sensor continuously monitors the DG operating temperature.
- The vibration sensor detects abnormal generator vibration.
- The ESP8266 successfully processes the sensor readings.
- The system automatically identifies normal and abnormal conditions.
- When an abnormal condition is detected, the buzzer and LED provide alerts.
- The relay can be used to demonstrate generator protection or shutdown.
- The system displays a basic analytics result such as **Healthy Operation** or **Maintenance Required**.

Results

The temperature sensor successfully monitors DG temperature.

The vibration sensor successfully detects abnormal vibration.

The ESP8266 processes and analyzes the sensor readings.

The system continuously monitors generator health.

Abnormal operating conditions are automatically detected.

The buzzer provides an audible warning.

The LED provides a visual indication.

The relay demonstrates generator protection.

The system displays the generator status and maintenance requirement.

The prototype demonstrates the basic operation of an **Industrial DG Monitoring and Analytics System**.

Performance Analysis:

Parameter	Expected Performance
Temperature Monitoring	Continuous
Vibration Monitoring	Continuous
Data Processing	Real-time
Condition Analysis	Automatic
Abnormal Condition Detection	Automatic
Maintenance Alert	Buzzer ON
Visual Warning	LED ON
DG Protection	Relay ON
Normal Condition	Buzzer and LED OFF

